

Artemis Student Platform (Eventer)

Този анализ е базиран само на файловете в репозиторието. Няма отделен API слой, няма requirements.txt и няма backend за Events.

1. Обобщение: маршрути, таблици и функционалности

Пути (всички в app.py, blueprint pages)

Метод	Път	Автентикация	Отговор	Поведение
GET	/	Няма	HTML	Начална страница
GET	/register	Няма	HTML	Форма за регистрация
POST	/register	Няма	HTML или JSON	Създава потребител; JSON връща 201 + JWT
GET	/login	Няма	HTML	Форма за вход
POST	/login	Няма	HTML или JSON	Проверява данните; задава session; JSON връща JWT
GET	/api/me	Bearer JWT (token_required)	JSON	Връща { id, email, role }
GET	/api/admin	Bearer JWT + роля admin	JSON	Връща { message: "Admin access granted." }
GET	/logout	Няма	Пренасочване	Изчиства session; пренасочва към /

JSON vs HTML се определя от:

App.py Lines 44-46

```
def wants_json_response():
```

```
return request.is_json or "application/json" in request.headers.get("Accept", "")
```

В Postman трябва да изпратите Content-Type: application/json **или** заглавка Accept: application/json, за да получите JSON отговор от `/register` и `/login`.

База данни

Файл	Таблица	Колони
<code>users.db</code> (SQLite)	<code>users</code>	<code>id</code> , <code>email</code> (UNIQUE), <code>password_hash</code> , <code>role</code> (по подразбиране 'student'), <code>created_at</code>

Няма таблици `events`, `clubs` или други.

App.py Lines 29-36

```
CREATE TABLE IF NOT EXISTS users (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
email TEXT UNIQUE NOT NULL,  
password_hash TEXT NOT NULL,  
role TEXT NOT NULL DEFAULT 'student',  
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
)
```

Има миграционна логика — добавя колона `role` чрез `ALTER TABLE`, ако липсва в по-стара база.

Работещи функционалности

Функционалност	Статус	Бележки
Регистрация (форма)	Работи	Email + парола; мин. 8 символа; regex за email
Регистрация (JSON)	Работи	Връща <code>token</code> , <code>role</code> ; опционално поле <code>role</code>
Вход (форма)	Работи	Задава <code>session["user_id"]</code> , <code>session["role"]</code>
Вход (JSON)	Работи	Връща JWT (валидност 60 мин.)
Защитен <code>/api/me</code>	Работи	Изисква Bearer token
Защитен <code>/api/admin</code>	Работи	Само роля <code>admin</code> → 403 за <code>student</code>
Изход (session)	Работи	Изчиства Flask session (JWT не се анулира)
Хеширане на пароли	Работи	<code>bcrypt</code>
Валидация на email	Работи	Regex на сървъра
Валидация на парола	Работи	Минимум 8 символа

Защита на session бисквитка	Частично	HttpOnly, SameSite=Lax, Secure в production
Превключване EN/BG	Само клиент	script.js
Начална страница	Работи	Статичен hero + bubble анимация

Само UI / не е имплементирано

UI елемент	Backend
Навигация Events / Clubs / About (href="#")	Няма
Бутони Explore Events / Join a Club	Няма
name, class във формата за регистрация	Събират се в HTML, не се записват
Показване на грешка при вход	Сървърът подава error, но login.html няма блок {% if error %}

Зависимости (импортирани, без requirements.txt)

- flask
- bcrypt
- PyJWT (import jwt)

2. Crud Testing

Не може да се изпълнят реални Event CRUD тестове

3. Задължителна текстова документация

3.1 Тестов план (стратегия W1 & W2)

Обхват

В обхвата (съществува днес):

- Автентикация чрез форма ([/register](#), [/login](#), /logout)
- JSON автентикация (POST /register, POST /login с JSON)
- JWT жизнен цикъл (издаване, употреба, изтичане, невалиден token)
- RBAC (/api/me, /api/admin)
- Поведение на SQLite таблица users
- Валидация на вход (email, дължина на парола, дублиран email)

Извън обхвата (липсва в кода):

- Event CRUD
- Страници Clubs, About
- Отнемане / refresh на token
- Автоматизиран тестов suite

W1 — Настройка, автентикация и Postman основа

Ден	Дейност	Резултат
-----	---------	----------

1	Инсталиране на зависимости (flask, bcrypt, PyJWT); задаване на SECRET_KEY; стартиране	Работещ сървър на http://127.0.0.1:5000
2	Smoke тестове: GET /, /register, /login; проверка на static файлове	Smoke checklist
3	Happy path: форма register → login → logout; JSON register → login	Дневник pass/fail
4	JWT поток: запазване на token; /api/me; липсващ/невалиден token	JWT дневник
5	Postman колекция + environment; експорт	Postman v1

Критерии за край на W1: Всички auth маршрути работят; JWT се издава и приема на /api/me; дублиран/невалиден вход връща правилни HTTP кодове.

W2 — RBAC, сигурност и регресия

Ден	Дейност	Резултат
1	RBAC: student vs admin на /api/admin; self-register като admin	RBAC дневник
2	Security checklist (раздел 3.5)	Попълнен checklist
3	Edge cases: изтекъл token, изтрит потребител, кратка парола, невалиден email	Edge-case дневник
4	UI/template пропуски: скрити login грешки; незаписани register полета	Доклад за дефекти
5	Пълна auth регресия + финален sign-off	Тестов доклад

Критерии за край на W2: Сигурностните находки са документирани; auth + JWT регресия е зелена; липсата на Event CRUD е официално записана.

Тестова среда

Променлива	Препоръчителна стойност
base_url	http://127.0.0.1:5000
SECRET_KEY	Силен случаен низ (избягвайте dev default при реално тестване)

FLASK_DEBUG	0 (освен при debug)
FLASK_ENV	production (за тест на Secure cookie флаг)
JWT валидност	60 минути (JWT_EXPIRY_MINUTES)

3.2 Postman workspace и environment конфигурация

Environment: Eventer Local

Променлива	Начална стойност	Описание
base_url	http://127.0.0.1:5000	Flask сървър
student_email	student@test.edu	Student акаунт
student_password	TestPass123!	Отговаря на мин. 8 символа
admin_email	admin@test.edu	Admin акаунт
admin_password	AdminPass123!	Admin парола
auth_token	(празно)	От login/register отговор
student_token	(празно)	Student JWT
admin_token	(празно)	Admin JWT

Структура на колекцията

Eventer - Artemis Platform

- ├─ Auth - JSON API
 - | └─ POST Register (Student)
 - | └─ POST Register (Duplicate → 409)
 - | └─ POST Register (Short Password → 400)
 - | └─ POST Register (Invalid Email → 400)
 - | └─ POST Login (Student)
 - | └─ POST Login (Invalid Credentials → 401)
 - | └─ GET Logout (Session only)
- ├─ Auth - Protected Routes
 - | └─ GET /api/me (Valid Token)
 - | └─ GET /api/me (No Token → 401)
 - | └─ GET /api/me (Invalid Token → 401)
 - | └─ GET /api/admin (Student Token → 403)
 - | └─ GET /api/admin (Admin Token → 200)
- ├─ Auth - HTML Forms (по избор)
 - | └─ GET Register Form
 - | └─ POST Register (Form)
 - | └─ GET Login Form
 - | └─ POST Login (Form + Cookie)
- └─ Events (БЛОКИРАНО)

└─ Няма endpoints

Примерни заявки

POST Register (JSON)

```
POST {{base_url}}/register
Content-Type: application/json
{
  "email": "{{student_email}}",
  "password": "{{student_password}}",
  "role": "student"
}
```

Tests script (запазване на token):

```
pm.test("Status 201 Created", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.test("Token returned", () => pm.expect(body.token).to.be.a("string"));
pm.environment.set("student_token", body.token);
pm.environment.set("auth_token", body.token);
```

POST Login (JSON)

```
POST {{base_url}}/login
Content-Type: application/json
{
  "email": "{{student_email}}",
  "password": "{{student_password}}"
}
```

GET /api/me

```
GET {{base_url}}/api/me
Authorization: Bearer {{auth_token}}
```

GET /api/admin

```
GET {{base_url}}/api/admin
Authorization: Bearer {{admin_token}}
```

Предварителна стъпка: създаване на admin

Регистрация чрез JSON с "role": "admin" (възможно днес — виж сигурност):

```
{
  "email": "{{admin_email}}",
  "password": "{{admin_password}}",
  "role": "admin"
}
```

3.3 Auth тестови случаи (6+)

ID	Заглавие	Стъпки	Очакван резултат
AUTH-01	JSON регистрация (student)	POST /register JSON { email, password }	201; body съдържа token,

			role: "student"
AUTH-02	JSON дублиран email	Регистрация с един и същ email два пъти	Втори опит: 409; { "error": "Email already exists." } }
AUTH-03	JSON кратка парола	POST с парола < 8 символа	400; грешка за дължина
AUTH-04	JSON невалиден email	POST email: "not-an-email"	400; грешка за email
AUTH-05	JSON успешен вход	POST /login с валидни данни	200; { token, role }
AUTH-06	JSON неуспешен вход	Грешна парола	401; { "error": "Invalid email or password." } }
AUTH-07	GET /api/me с валиден token	Bearer token от login	200; { id, email, role }
AUTH-08	GET /api/me без token	Без Authorization header	401; { "error": "Missing bearer token." } }
AUTH-09	GET /api/me с невалиден token	Bearer invalid.token.here	401; { "error": "Invalid token." } }
AUTH-10	GET /api/admin като student	Student JWT на /api/admin	403; { "error": "Forbidden: insufficient role." } }
AUTH-11	GET /api/admin като admin	Admin JWT на /api/admin	200; { "message": "Admin access granted." } }
AUTH-12	Form register + redirect	POST form към /register	302 → /login
AUTH-13	Form login задава session	POST form към /login	302 → / ; session cookie зададен
AUTH-14	Logout изчиства session	GET /logout след form login	Session изчистен; redirect /
AUTH-15	Self-register като admin (сигурност)	JSON register с "role": "admin"	201; потребител с admin роля

3.4 Event CRUD тестови случаи (4+)

Статус: НЕИЗПЪЛНИМИ — backend липсва.

ID	Заглавие	Статус	Причина
----	----------	--------	---------

EVT-01	Създаване на събитие	БЛОКИРАН	Няма POST маршрут или таблица events
EVT-02	Списък със събития	БЛОКИРАН	Няма GET маршрут
EVT-03	Редакция на събитие	БЛОКИРАН	Няма PUT/PATCH маршрут
EVT-04	Изтриване на събитие	БЛОКИРАН	Няма DELETE маршрут

Само UI (не е CRUD):

ID	Заглавие	Стъпки	Очаквано
EVT-UI-01	Events навигация	Клик „Events“ на /	Няма навигация (href="#")
EVT-UI-02	Explore Events CTA	Клик на бутона на /	Няма навигация (href="#")

За да се изпълни заданието, все още са нужни:

- таблица events
- маршрути напр. GET/POST /api/events, GET/PUT/DELETE /api/events/<id>
- аутентикация при променящи операции

3.5 Security checklist (документация)

Аутентикация и токени

Проверка	Статус	Находка
Хеширани пароли (bcrypt)	PASS	hash_password() / verify_password()
Parameterized SQL	PASS	? placeholders навсякъде
JWT подпис (HS256)	PASS	60-минутна валидност
Валидация на email формат	PASS	EMAIL_RE regex
Мин. дължина на парола (8)	PASS	Server-side при register
Bearer token на защитени маршрути	PASS	/api/me, /api/admin
Role check на admin маршрут	PASS	roles_required("admin")
Self-assign admin при register	FAIL	JSON "role": "admin" е позволен
JWT revocation при logout	FAIL	Logout изчиства само session; JWT валиден до изтичане
Rate limiting / brute-force защита	FAIL	Неограничени login опити

CSRF на HTML форми	FAIL	Няма CSRF tokens
--------------------	-------------	------------------

Session и конфигурация

Проверка	Статус	Находка
Secret от environment	ЧАСТИЧНО	SECRET_KEY / JWT_SECRET_KEY; fallback "dev-only-change-this-secret"
Session HttpOnly	PASS	SESSION_COOKIE_HTTPONLY = True
Session SameSite	PASS	Lax
Session Secure (prod)	PASS	При FLASK_ENV=production
Debug режим	ПОДОБРЕН	debug само ако FLASK_DEBUG=1
requirements.txt / фиксирани deps	FAIL	Липсва

Авторизация и данни

Проверка	Статус	Находка
Защитени HTML страници	N/A	/ е публична; session login не ограничава HTML
Event ownership / ACL	НЕИМПЛЕМЕНТИРАНО	Няма events
Register profile полета (name, class)	FAIL	UI ги събира; сървърът ги игнорира
Login error показван на потребителя	FAIL	login.html няма error блок

JWT payload

JWT съдържа: sub (user id), email, role, iat, exp. Потребителят се проверява от DB при всяка защитена заявка (добре — изтрити потребители получават 401).

4. Липсваща функционалност (блокира заданието)

Изискване	Налице?	Пропуск
Event CRUD (4+ теста)	Не	Няма таблица, маршрути, API
UI страници за събития	Не	Само # placeholder връзки
Token refresh / revoke	Не	Само stateless JWT
Контрол на admin provisioning	Не	Всеки може да се регистрира като admin
Автоматизирани тестове	Не	Няма pytest/unittest
requirements.txt	Не	Ръчна инсталация
Login error UX	Частично	Template пропуск

Запазване на profile полета	Не	name/class се отхвърлят
-----------------------------	----	-------------------------

5. Слабости в сигурността и пропуски в тестването

Висок приоритет

1. **Privilege escalation при регистрация** — "role": "admin" се приема на публичен `/register`.
2. **Няма JWT invalidation** — откраднат token работи до изтичане, дори след logout.
3. **Dev secret по подразбиране** — слаб, ако SECRET_KEY не е зададен в env.

Среден приоритет

4. **Няма CSRF защита** на HTML форми.
5. **Няма rate limiting** на login/register.
6. **Двоен auth модел** — session + JWT; HTML маршрутите не ползват JWT.
7. **Несъответствие в register формата** — задължителни name/class не се записват.

Пропуски в тестването

Област	Пропуск
Event CRUD	Функционалността липсва — основен блокер
Изтекъл token	Изисква 60+ мин. или mock на времето
Token след изтриване на user	Тествуемо чрез DB delete + <code>/api/me</code>
Session vs JWT parity	Form login не помага на JSON-only защитени маршрути без отделен token call
Automated CI	Липсва
Reproducible dependencies	Няма lockfile